SYSTEM DESIGN

Two-Phase Commit Visual Guide

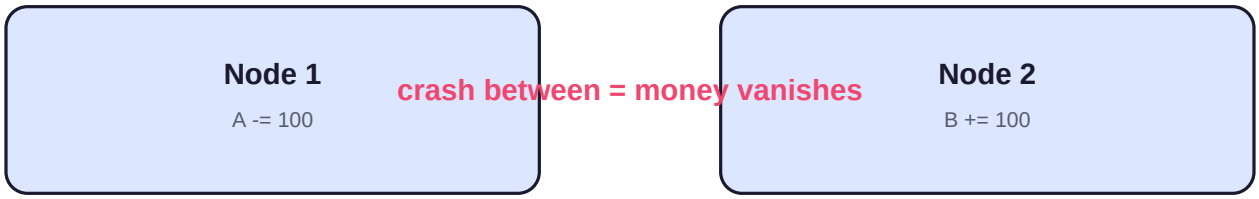
Atomic commits across nodes:
prepare, decide, and the blocking flaw.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Atomicity Need

Move $100 from A (node 1) to B (node 2): both updates or NEITHER.

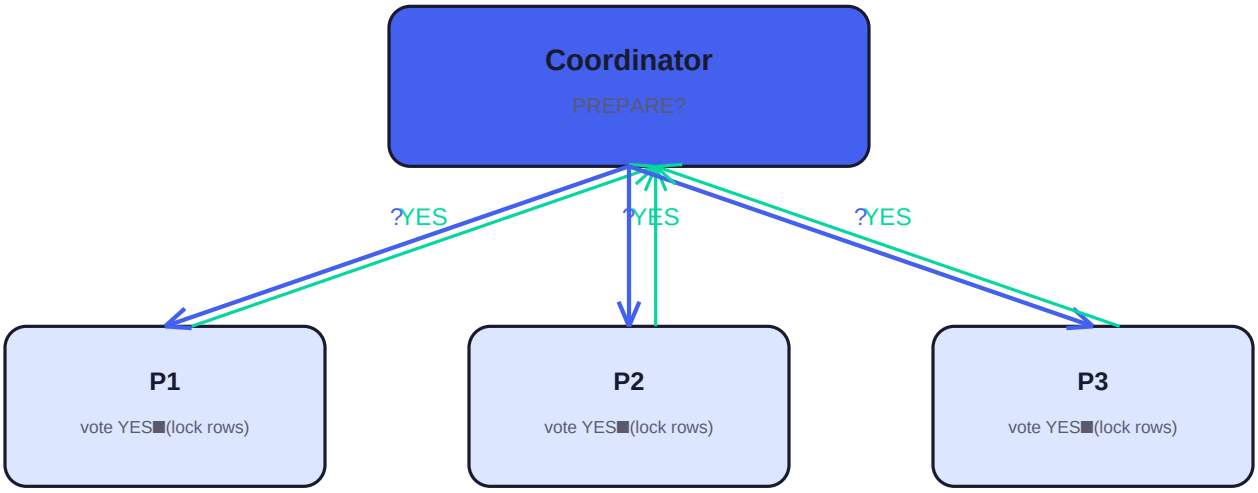


- One node committing while the other aborts = corruption no retry can fix.
- 2PC's answer: a COORDINATOR runs a vote, then tells everyone the unanimous verdict.
- Assumption: nodes fail by crashing (not lying) -- this is crash-fault atomic commit.

STEP 2/6

Phase 1: Prepare (Voting)

Coordinator: 'can everyone commit?' Each participant votes YES or NO.

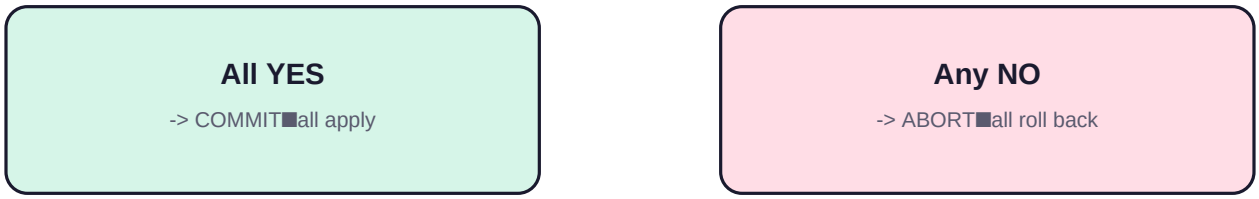


Voting YES means: 'I have locked everything and PROMISE to commit if told.' Locks are held from here on.

STEP 3/6

Phase 2: Commit or Abort

Unanimous YES -> COMMIT everywhere. Any NO (or timeout) -> ABORT everywhere.

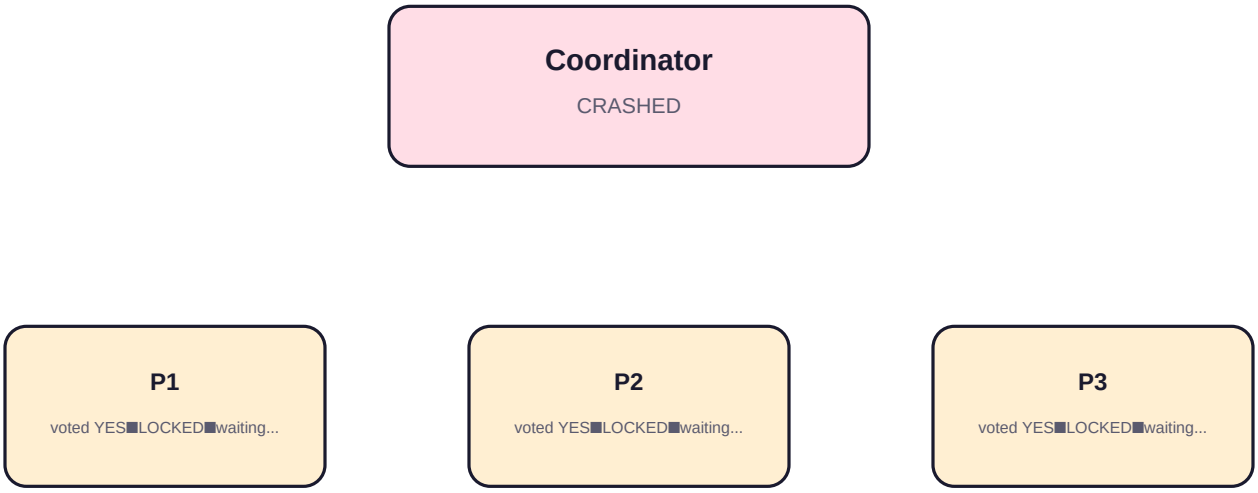


- The coordinator's decision is written to stable storage BEFORE announcing -- it must survive its own crash.
- Participants that voted YES must OBEY the verdict: that promise is what makes it atomic.
- Once decided, the outcome is inevitable: stragglers learn it on recovery and comply.

STEP 4/6

The Blocking Problem

Coordinator crashes after votes, before the verdict: participants wait FOREVER.

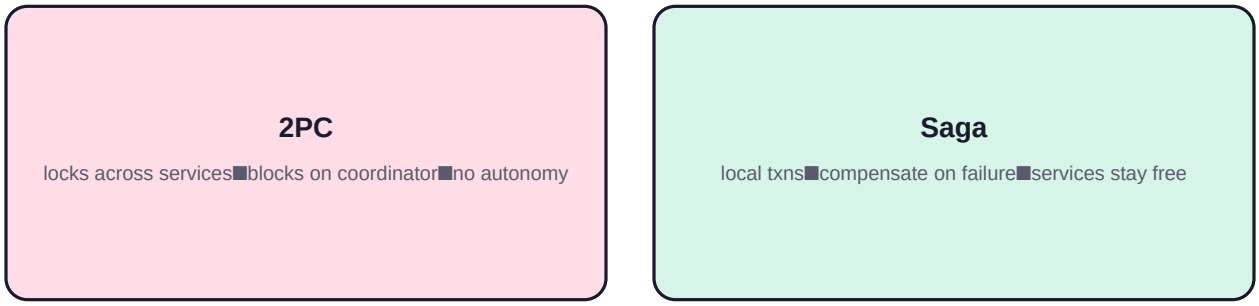


- Locked rows stay locked: no other transaction can touch them. Throughput dies, not just this txn.
- Participants cannot unilaterally decide: a peer might have heard COMMIT before the crash.
- Recovery needs the coordinator's log -- or a human. This BLOCKING is 2PC's fatal flaw.

STEP 5/6

Why Sagas Replaced It

Availability beats atomicity when the participants are independent services.



- Microservices own their data: holding their rows locked for a stranger's coordinator is unacceptable.
- Saga trade: give up isolation (others see partial progress), gain availability (nothing blocks).
- 3PC adds a pre-commit phase to reduce blocking -- but cannot eliminate it (and adds messages).

STEP 6/6

Where 2PC Still Lives

Narrow, controlled, single-vendor -- the places blocking is tolerable.

- Single-database XA: Postgres/MySQL as participants under one transaction manager -- same trust domain.
- Spanner-style: 2PC OVER Paxos -- the coordinator is replicated, so 'coordinator crash' stops blocking.
- Rule: 2PC is fine when participants and coordinator share fate (one team, one DC, replicated log).
- Across companies, clouds, or autonomous services: saga. Every time.

STEP ✓I✓

Key Takeaways

Two-Phase Commit — Remember

- 2PC gives atomic commit across crash-prone nodes: prepare/vote, then commit/abort on unanimous verdict.
- Voting YES is a promise: rows stay locked until the coordinator's decision arrives.
- The fatal flaw is blocking: a dead coordinator leaves participants locked indefinitely.
- Sagas replaced 2PC across services -- local transactions plus compensation, no global locks.
- 2PC survives where coordinator and participants share fate: single-DB XA and replicated coordinators.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev